PrepLang: data-aware compilation and guarded reuse of data-preparation pipelines

Nikhil V[a,*], Ranjithkumar S[b]

[a]*School of Computer Science and Engineering, Vellore Institute of Technology, Vellore 632014, Tamil Nadu, India*
[b]*Department of Analytics, School of Computer Science and Engineering, Vellore Institute of Technology, Vellore 632014, Tamil Nadu, India*

Abstract

PrepLang is an open-source compiler for tabular and image preparation. It analyzes a pipeline together with an available input batch, using CSV values, Parquet metadata and image headers. The analysis discharges contracts it can prove, reports definite violations and retains unresolved checks for execution. The compiler emits Python code for pandas and OpenCV and records the input facts needed by each speculative removal. On a later batch, the generated program checks these facts and restores affected instructions; it refuses inputs outside its admission conditions. In 426 pipeline–batch trials across five public datasets, using first-month and previous-month compilation anchors, reuse agreed with fresh compilation on outputs or contract failures. Differential testing also found no disagreement for 8,000 generated tabular programs. Lean proofs establish correctness for a restricted language core; the C++ implementation is tested, not formally verified. The software includes a pandas-prefix translator, diagnostics, examples and scripts for inspecting and reproducing the evaluation.

Keywords: domain-specific language, data-aware compilation, abstract interpretation, speculative optimization, deoptimization, data validation

Metadata

1. Motivation and significance

Data preparation makes assumptions that are often absent from a script's interface: a column contains numbers, a value lies within a range, or an image

*Corresponding author
Email addresses: atnikhil06@gmail.com (Nikhil V), ranjithkumar.s@vit.ac.in (Ranjithkumar S)

Nr.	Code metadata description	Value
C1	Current code version	v2.1.0
C2	Permanent link to code/repository used for this code version	`https://github.com/Nikhi00718/preLang/tree/v2.1.0`
C3	Permanent link to Reproducible Capsule	Docker image `ghcr.io/nikhi00718/preplang:2.1.0`, built from the `Dockerfile` of the release; `eval/reproduce.sh` in C2 regenerates or reruns the evaluation (Section 4).
C4	Legal Code License	MIT
C5	Code versioning system used	git
C6	Software code languages, tools, and services used	C++17 (standard library only); Python 3 with pandas, NumPy, PyArrow, OpenCV, matplotlib; Lean 4; CMake
C7	Compilation requirements, operating environments and dependencies	CMake 3.16+, a C++17 compiler (CI configurations: Visual Studio 2022 on Windows, GCC on Ubuntu and macOS); Python 3.12+ recommended, pandas 2.0+, NumPy and PyArrow; OpenCV for images, matplotlib for charts; Lean 4.34.1 for the optional proof build. Evaluation versions are listed in Section 4.
C8	If available, link to developer documentation/manual	`README.md` (user manual), `LANGUAGE.md` (reference), `ARCHITECTURE.md`, `CONTRIBUTING.md`; the READMEs of `eval/`, `paper/`, `proofs/`, `docs/`
C9	Support email for questions	atnikhil06@gmail.com

Table 1: Code metadata. C2 identifies the software and stored results evaluated in this article.

operation preserves a usable output type. Such assumptions contribute to the maintenance cost of machine-learning systems [1]. A pandas [2] or OpenCV [3] script may discover a violation only after processing has started. Validators such as TFDV [4], Deequ [5], Great Expectations [6] and pandera [7] check datasets against constraints. They do not, by themselves, propagate those constraints through a program and generate guarded optimizations.

PrepLang [8] targets batch workflows in which a representative input is available before the pipeline runs. Its compiler scans CSV values, reads Parquet metadata and inspects image headers, then propagates an abstract description of the data through the program. A contract is proven, definitely violated, or unresolved: proven checks can be removed, violations are diagnosed, and unresolved checks remain in the generated program.

The complication is reuse. A check that is redundant for one batch may

Table 2: PrepLang and related tools. *Values*: missing values and value ranges.

Tool	Decides at	Knows about the data	Follows code	On a new batch
TFDV [4], Deequ [5], GX [6], pandera [7]	run time	values, from each batch	no	validates it again
F# type providers [9]	compile time	types, from a sample	no	–
Tuplex [13]	compile time	types and cases, from a sample	yes	falls back per exceptional row
Zhuang and Lu [15]	compile time	column types, from the code	yes	–
Gerstl et al. [16]	compile time	R data-frame shapes, optionally from the data	yes	–
Mirji [17]	compile time	declared schemas	yes	checks the output schema
Schelter et al. [14]	development time, by running it	record-level provenance	yes	–
PrepLang	**compile time**	**types and values of every cell; image headers**	**yes**	**admits by certificate; reruns only the removals it breaks**

be necessary for the next. For each speculative removal, PrepLang records an *item certificate*, the input facts that justify it. The generated program checks the certificates on later batches and restores an instruction when its certificate fails. A separate *admission certificate* holds requirements for which the program has no fallback, such as an incompatible column type; failing it stops execution. A changed value range therefore restores one check without recompiling the rest of the pipeline.

The underlying ideas have precedents (Table 2). Type providers derive types from external data [9]; soft constraints use properties observed in a database for query optimization [10]; dynamic deoptimization restores execution when speculative assumptions fail [11, 12]; Tuplex compiles a common path from sampled data with fallbacks for exceptions [13]; and provenance-based screening runs a preparation pipeline to catch issues such as train/test leakage [14]. PrepLang combines these ideas in a small preparation language whose certificates track the value facts behind individual removals. It does not claim to have introduced data-dependent optimization or deoptimization.

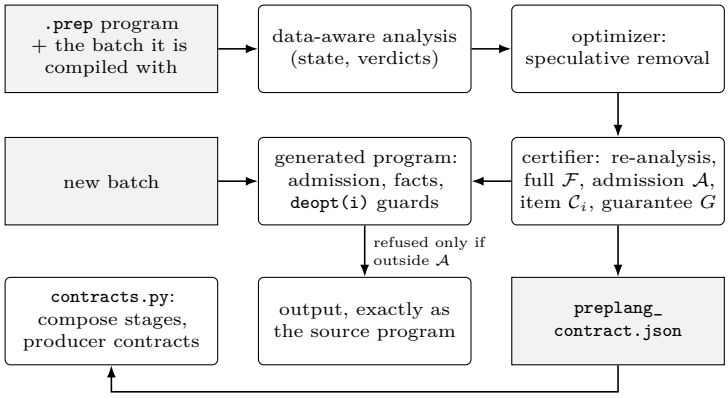


Figure 1: How PrepLang compiles once and reuses the result. The certifier re-runs the analysis with weaker input facts, reading no data; on a new batch, the generated program runs a removed instruction i only if the batch breaks its item certificate $\mathcal{C}_i$.

2. Software description

2.1. Software architecture

The C++17 compiler uses only the standard library and has seven phases: lexing, parsing, semantic analysis, intermediate representation, optimization, certification and Python code generation (Fig. 1). Semantic analysis tracks types, missing values, numeric intervals and image properties at each statement. Parquet analysis uses footer statistics and image analysis reads headers, so some facts stay unknown and the relevant checks are kept. After optimization the certifier weakens input facts and reanalyzes the program, without rereading the input, to find what each certificate must contain. The output is one Python program, which checks admission and guards speculative removals with `deopt(i)`, and a machine-readable `preplang_contract.json`.

The Lean 4 development [18] proves selective deoptimization correct for any sound, monotone analysis, and establishes those assumptions for a core of numeric columns, missing values, interval filters, null filling, addition and range/not-null contracts. The C++ compiler, its parsers, pandas and OpenCV are outside the proof, as are joins, aggregation, dates and images; differential tests cover joins and aggregation. The repository contains 316 C++ checks in 10 suites, integration tests and parser fuzz targets, run in CI on Windows, Ubuntu and macOS with sanitizers, a coverage gate and the Lean build.

2.2. Software functionalities

A program loads CSV or Parquet tables, selects and derives columns, filters rows, fills or drops missing values, joins on one key and aggregates groups. Contracts cover types, nullability, ranges, uniqueness and references;

image pipelines check channels and dimensions, link labels, resize and convert. Diagnostics give a source location and, where available, a witness row and a hint. Optimization removes proven checks and redundant operations, prunes unread columns and streams images; rewrites that may change the output need `--approx`, and the exactness results here exclude them. Every phase can be printed, with an execution plan and JSON diagnostics, and `PLOT` and `REPORT` chart a batch. A translator lifts the supported prefix of a pandas script, and with `--facts` a program compiles from differentially private facts released by a data owner, while run-time checks still read the real data. PrepLang installs with `pip install .`; `README.md`, `LANGUAGE.md` and `ARCHITECTURE.md` document installation, the language and the implementation.

3. Illustrative examples

Consider this program with the eight-row `examples/tabular/students.csv`:

```
LOAD "students.csv" AS students;
EXPECT students.age NUMBER;
EXPECT students.marks BETWEEN 0 AND 100;
EXPECT students.name NOT_NULL;
FILTER students WHERE marks >= 50;
EXPECT students.marks BETWEEN 60 AND 100;
SAVE students AS "clean.csv";
```

The compiler proves the first three contracts. After the filter it knows only that marks lie in $[50, 95]$, so the last check remains at run time. The range and not-null checks are removed speculatively. On a new batch containing a mark of 104, the generated program restores the line-3 check and rejects the batch, as a fresh compilation does (there, during compilation). A batch within the certificates runs with both checks removed; text in the numeric `age` column fails admission. For images, `examples/images/linked.prep` scales pixels to $[0, 1]$ and saves arrays. Requesting PNG output for those floating-point pixels gives a compile-time type error before any pixel is decoded; the equivalent OpenCV script writes almost-black images without an error.

4. Impact

PrepLang began as a compiler-design project and has no independent deployment or user study yet. Its evidence is the stored evaluation in `eval/results/`, from which every number below is generated.

Table 3: Main results of the evaluation (stored in `eval/results/`).

Experiment	Result
Differential test, 4 seeds	8,000 random programs (1,018 with many-to-many joins) agree with an independent interpreter, with and without optimization; 0 of 1,569 and 4,814 reuses wrong
Numeric type changes	1,911 batches: 1,341 agree, 570 refused (57 of them fail fresh compilation too), 0 wrong
Five monthly pipelines	426 pipeline–batch trials agree with fresh compilation, 17 of them on a contract failure; 39 restore an instruction
Validators as admission gates	GX rejects 409, pandera 407 executable trials; TFDV admits 10 that disagree
Lifted pandas scripts	378 of 733 Kaggle scripts lifted, 4131 of 4158 batches agree; 85 of 945 traffic and bike scripts, all 3710 months agree
GitHub preparation bugs	18 of 44 reported at compile time (6 need an `EXPECT`)
Census of 453 programs	157 changed by the optimizer, median speedup 1.07×, all outputs identical; compile time median 29 ms, maximum 2.4 s

Correctness and reuse. Across four seeds, 8,000 generated tabular programs were checked with and without optimization against a separate Python interpreter that shares no compiler code. Agreement means the same output values (within relative and absolute tolerances of 10^{-9}) or the same failing contract. All agreed, and no admitted reuse gave a wrong outcome (Table 3). Of the batches in which one numeric column changed between integer and float, 570 were refused. Most refusals are deliberate: the type rule only relaxes columns that no `DERIVE`, join key or aggregate combines with another column, and 400 of the 513 refused batches that a fresh compilation accepts changed such a combined column. The test shows that admitted batches are handled correctly, not that admission accepts every safe batch.

Five authored pipelines use NYC green taxi, Citi Bike Jersey City, and UCI traffic, bike-sharing and Beijing air-quality data. Each is compiled on the first month, or on the preceding month, and run on the later ones: 426 pipeline–batch trials, with target months repeated across the two schedules. All agreed with fresh compilation, 17 of them on a contract failure. Fresh compilation shares the implementation under test, so this checks reuse consistency; the independent interpreter above checks correctness.

Coverage and cost. The translator lifts only supported prefixes, and the script corpora are tied to particular datasets, so the coverage figures in Table 3 do not describe Python programs in general. Speed is not PrepLang's main benefit: on a synthetic 200,000-row table the optimized program took 2.32 s

against 2.49 s for a script (median of three runs, including Python start-up, on an Intel Core i5-12450H with Windows 11, pandas 3.0 and Python 3.14), and the census median speedup is 1.07×. Compilation is paid once per pipeline, not per batch: admitting a later batch by its certificate changed the run time by −1% to +2%. The benefit lies in errors reported before data is processed and in batches that run without recompilation, with removed checks restored only where needed.

The validators (Great Expectations 0.18.22, pandera 0.34, TFDV 1.21) were used as admission gates inferred from the compiled batch. Their results measure those gates, not the tools' ability to enforce hand-written contracts. With private facts at $\varepsilon = 1$, compilation kept every removal in the taxi and traffic pipelines but 14% in the bike pipeline, against 54% for the AIM-based configuration [19]; no output of 64,203 later runs disagreed with fresh compilation.

Reproduction. The release in Table 1 pins the compiler and the stored results. `eval/reproduce.sh check` builds and tests the software and regenerates every number from those results; `full` reruns the experiments, including all four seeds and both lifter versions, and `baselines` the validators and private compilation. Kaggle inputs must be downloaded by hand; the GitHub corpora are pinned by commit and hash.

5. Conclusions

PrepLang makes data assumptions explicit before a preparation pipeline runs and keeps the conditions behind removed work when the compiled program is reused. Joins use one key, Python is the only backend, numeric type changes are admitted only for uncombined columns, and the proof covers a restricted core; the implementation is tested, not verified. Whether PrepLang pays off depends on whether a pipeline fits the language and is run on many batches.

6. Future plans

Joins on several key columns, code generation for other targets, and a study with users who write preparation scripts.

CRediT authorship contribution statement

Nikhil V: Conceptualization, Methodology, Software, Validation, Investigation, Data curation, Visualization, Writing – original draft. **Ranjithkumar S:** Supervision, Writing – review and editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Data availability

The source code, evaluation scripts and stored results are available in the v2.1.0 release in Table 1. PrepLang is distributed under the MIT license. External datasets and script corpora retain their original licenses and access conditions. Download instructions and corpus manifests are in `eval/README.md`; Kaggle inputs require manual access. The submission includes the user and developer documentation.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work, the authors used ChatGPT and Claude for implementation assistance, literature search, evaluation scripts, analysis, drafting and language editing. The authors reviewed and edited the resulting content and take full responsibility for the published work.

References

[1] D. Sculley, G. Holt, D. Golovin, E. Davydov, T. Phillips, D. Ebner, V. Chaudhary, M. Young, J.-F. Crespo, and D. Dennison, "Hidden technical debt in machine learning systems," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2015, pp. 2503–2511.

[2] W. McKinney, "Data structures for statistical computing in Python," in *Proc. 9th Python in Science Conf. (SciPy)*, 2010, pp. 56–61. `https://doi.org/10.25080/Majora-92bf1922-00a`.

[3] G. Bradski, "The OpenCV library," *Dr. Dobb's Journal of Software Tools*, 2000.

[4] E. Breck, N. Polyzotis, S. Roy, S. E. Whang, and M. Zinkevich, "Data validation for machine learning," in *Proc. Conf. on Machine Learning and Systems (MLSys)*, 2019.

[5] S. Schelter, D. Lange, P. Schmidt, M. Celikel, F. Biessmann, and A. Grafberger, "Automating large-scale data quality verification," *Proc. VLDB Endowment*, vol. 11, no. 12, pp. 1781–1794, 2018. https://doi.org/10.14778/3229863.3229867.

[6] Great Expectations, "Great Expectations: data quality testing," https://greatexpectations.io.

[7] N. Bantilan, "pandera: Statistical data validation of pandas dataframes," in *Proc. 19th Python in Science Conf. (SciPy)*, 2020, pp. 116–124. https://doi.org/10.25080/Majora-342d178e-021.

[8] Nikhil V, PrepLang, version 2.1.0 [Computer software], GitHub, 2026. https://github.com/Nikhi00718/preLang/tree/v2.1.0.

[9] T. Petricek, G. Guerra, and D. Syme, "Types from data: Making structured data first-class citizens in F#," in *Proc. 37th ACM SIGPLAN Conf. on Programming Language Design and Implementation (PLDI)*, 2016, pp. 477–490. https://doi.org/10.1145/2908080.2908115.

[10] P. Godfrey, J. Gryz, and C. Zuzarte, "Exploiting constraint-like data characterizations in query optimization," in *Proc. ACM SIGMOD Int. Conf. on Management of Data*, 2001, pp. 582–592. https://doi.org/10.1145/375663.375749.

[11] U. Hölzle, C. Chambers, and D. Ungar, "Debugging optimized code with dynamic deoptimization," in *Proc. ACM SIGPLAN Conf. on Programming Language Design and Implementation (PLDI)*, 1992, pp. 32–43. https://doi.org/10.1145/143103.143114.

[12] A. Barrière, S. Blazy, O. Flückiger, D. Pichardie, and J. Vitek, "Formally verified speculation and deoptimization in a JIT compiler," *Proc. ACM Program. Lang.*, vol. 5, no. POPL, pp. 1–26, 2021. https://doi.org/10.1145/3434327.

[13] L. Spiegelberg, R. Yesantharao, M. Schwarzkopf, and T. Kraska, "Tuplex: Data science in Python at native code speed," in *Proc. ACM SIGMOD Int. Conf. on Management of Data*, 2021, pp. 1718–1731. https://doi.org/10.1145/3448016.3457244.

[14] S. Schelter, S. Guha, and S. Grafberger, "Automated provenance-based screening of ML data preparation pipelines," *Datenbank-Spektrum*, vol. 24, no. 3, pp. 187–196, 2024. https://doi.org/10.1007/s13222-024-00483-4.

[15] Y. Y. Zhuang and M.-Y. Lu, "Enabling type checking on columns in data frame libraries by abstract interpretation," *IEEE Access*, vol. 10, 2022. https://doi.org/10.1109/ACCESS.2022.3146287.

[16] O. Gerstl, F. Sihler, and M. Tichy, "Inferring the shape of data frames in R programs using abstract interpretation," arXiv preprint arXiv:2607.03889, 2026. https://doi.org/10.48550/arXiv.2607.03889.

[17] V. Mirji, "Shift schema drift left: Policy-aware compile-time contracts for typed JVM and Spark pipelines," arXiv preprint arXiv:2604.16986, 2026. https://doi.org/10.48550/arXiv.2604.16986.

[18] L. de Moura and S. Ullrich, "The Lean 4 theorem prover and programming language," in *Proc. 28th Int. Conf. on Automated Deduction (CADE)*, LNCS 12699, 2021, pp. 625–635. https://doi.org/10.1007/978-3-030-79876-5_37.

[19] R. McKenna, B. Mullins, D. Sheldon, and G. Miklau, "AIM: An adaptive and iterative mechanism for differentially private synthetic data," *Proc. VLDB Endowment*, vol. 15, no. 11, pp. 2599–2612, 2022. https://doi.org/10.14778/3551793.3551817.